

LinkQ: An LLM-Assisted Visual Interface for Knowledge Graph Question-Answering

Harry Li*
MIT Lincoln Laboratory

Gabriel Appleby†
Tufts University

Ashley Suh‡
MIT Lincoln Laboratory

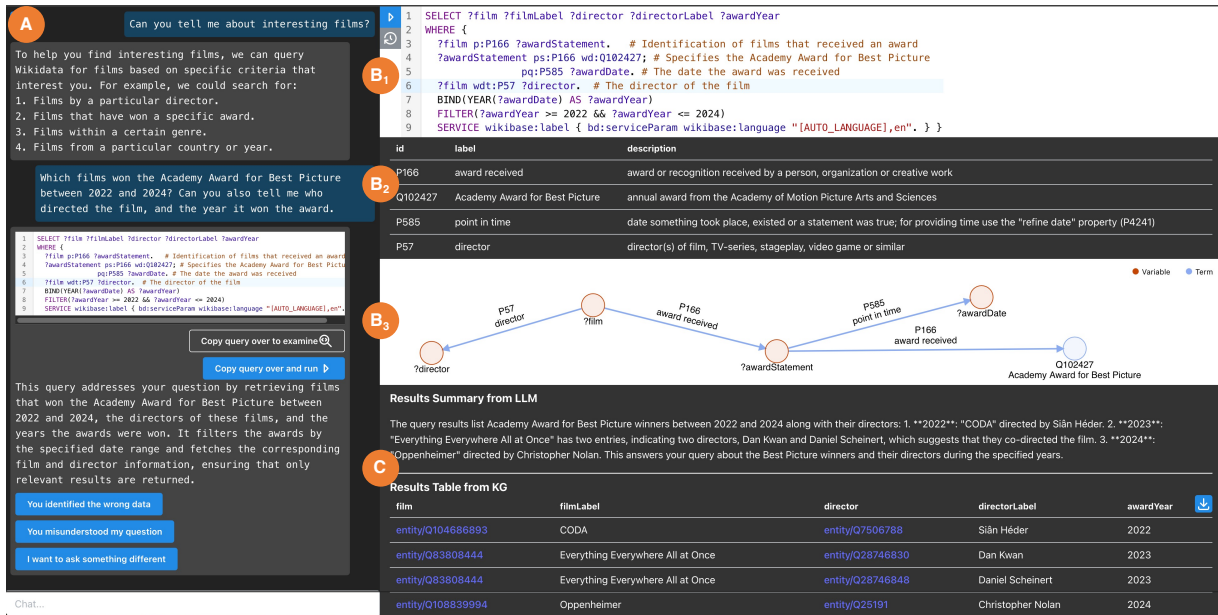


Figure 1: Exemplar workflow for LinkQ, a system leveraging an LLM for refining natural language questions into knowledge graph queries. The (A) *Chat Panel* lets users communicate with the LLM to ask specific or open-ended questions. The *Query Preview Panel* consists of three components: the (B1) *Query Editor*, which supports interactive editing; the (B2) *Entity-Relation Table*, which provides mapped data IDs from the KG, helping to assess the correctness of the LLM’s generated query; and the (B3) *Query Graph*, which visualizes the structure of the query to illustrate the underlying schema of the KG. Finally, the (C) *Results Panel* provides a cleaned, exportable table as well as an LLM-generated summary based on the query results. Importantly, LinkQ ensures all data retrieved and summarized by the LLM comes from ground truth in the KG.

ABSTRACT

We present LinkQ, a system that leverages a large language model (LLM) to facilitate knowledge graph (KG) query construction through natural language question-answering. Traditional approaches often require detailed knowledge of a graph querying language, limiting the ability for users – even experts – to acquire valuable insights from KGs. LinkQ simplifies this process by implementing a multistep protocol in which the LLM interprets a user’s question, then systematically converts it into a well-formed query. LinkQ helps users iteratively refine any open-ended questions into precise ones, supporting both targeted and exploratory analysis. Further, LinkQ guards against the LLM *hallucinating* outputs by ensuring users’ questions are only ever answered from ground truth KG data. We demonstrate the efficacy of LinkQ through a qualitative study with five KG practitioners. Our results indicate that practitioners find LinkQ effective for KG question-answering, and desire future LLM-assisted exploratory data analysis systems.

*e-mail: harry.li@ll.mit.edu

†e-mail: gabriel.appleby@tufts.edu

‡e-mail: ashley.suh@ll.mit.edu

Index Terms: Knowledge graphs, large language models, query construction, question-answering, natural language interfaces.

1 INTRODUCTION

Knowledge graphs (KGs) have become an industry standard for storing complex relationships in data for various tasks [3, 11], such as recommendation generation [10] and visualization creation [18]. Despite their widespread use for managing data, a significant barrier to KG effectiveness remains the retrieval of relevant data through querying [20, 19, 17]. This is partly because KG data retrieval can require a highly technical understanding of the KG itself or the KG’s querying language [23], which typically differs across different graph databases [15]. When the user is a non-expert in querying, they must rely on KG query builders or traditional NL interfaces [6, 9, 33, 8], which tend to limit the expressivity of the user’s data analysis questions due to their rule-based syntax [14]. Consequently, valuable insights contained in KGs are difficult to attain, limiting the accessibility of KGs in practice [17].

In this paper, we contribute LinkQ¹, a system that utilizes a large language model (LLM) conversational agent to assist users in exploring and answering questions about KG data. Recent work has shown that LLMs are capable of interpreting analytic questions from natural language [2, 34] and writing code [28], including

¹Open-source code: <https://github.com/mit-ll/linkq>

KG queries [28, 26]. LinkQ combines these ideas, implementing SPARQL-based targeted and exploratory question-answering with GPT-4 [24] for the WikiData [32] RDF knowledge graph [1]. When users ask open-ended questions about KG data, LinkQ guides the LLM in helping the user iteratively refine those questions until they can be translated into precise KG queries.

Integrating LLMs with KGs has emerged as a promising research direction for data management applications [7, 25]. KGs are largely used to help mitigate an LLM’s *hallucinations* [21, 31], that is, the generation of false, erroneous data [27]. A recent survey paper details current techniques in this space [4]. KGs can also help an LLM access information that it was not previously trained on, as KG data can be kept consistently up-to-date without incurring the expensive retraining process for LLMs [11, 32].

Building upon these ideas, we devise a protocol in LinkQ to ensure the LLM answers the user’s questions by constructing and executing KG queries. Our protocol is designed to mitigate the LLM from hallucinating false data during query construction (Figure 2), in its outputs (Section 3), and allows users to retrieve recent data that may not be present in industry-standard LLMs. For example, in Figure 1, the LLM (GPT-4) uses ground-truth KG data to correctly show the 2024 Best Picture winning film, which (at the time of this writing) is data it has not been trained on [24]. LinkQ implements an interactive graph query visualization and an entity-relation table (Section 4), which are populated with ground-truth data from the KG. Both of these visualizations are included to facilitate users in assessing the accuracy of the LLM’s query.

To demonstrate how LinkQ can be used for KG query building from natural language (NL), we conducted a qualitative study with five data scientists who regularly work with KGs (Section 5). In this demonstration, we walked through several use cases for the system and allowed participants to ask their own questions for exploratory analysis. Participants found high value in the workflow of LinkQ, particularly the LLM’s explanations for its query generation process, the query graph for showcasing KG connections, and the alleviated burden of constructing KG queries altogether. To conclude, we discuss the limitations of LinkQ and future directions for designing LLM-driven visual analysis tools for KGs (Section 6).

2 SYSTEM DESIGN GOALS

In the initial stages of designing LinkQ, we leveraged the learning outcomes from previous work in KG query builders [6, 9, 8], NL question-answering interfaces [29, 30, 22, 12], as well as the feedback we received over a month-long collaboration with three data scientists who regularly use KGs in their work. Altogether, we arrived at the following design goals for our system:

- G1 Support back-and-forth conversation to refine natural language questions into precise queries.** Users may have open-ended or targeted questions they want to ask about data in the KG. When the question is open-ended, an LLM should help users refine their search.
- G2 Mitigate the LLM’s tendency to hallucinate false information.** Using an LLM to generate KG queries may result in false data IDs or false query results. Precautions should be made to reduce the frequency of these occurrences.
- G3 Preview information about the LLM’s generated query to assess its accuracy.** KG queries can be computationally expensive to execute. A preview of the query, with relevant information, should be displayed to users to help assess the accuracy of both the data and the query before its execution.
- G4 Provide multimodal query results from both the LLM and the KG.** Users perform question-answering with KGs for a multitude of purposes. Therefore, query results should be displayed in text form (for consumption) as well as tabular form (for further analysis).



Figure 2: Example of how an LLM can *hallucinate* [27] or misrepresent the data in a KG. Here, we ask ChatGPT [5] to write a query about gold medal winners from the 2020 Summer Olympics. In total, ChatGPT wrote three consecutive failing queries with incorrect IDs, while stating to us that it had corrected its previous mistakes.

3 LINKQ

LinkQ follows a state-machine workflow, shown in Figure 3. We provide details on the exact prompts and algorithmic workflow used for LinkQ in our supplemental material and open-source code, which are available at <https://github.com/mit-11/linkq>.

An important distinction to make in LinkQ is the difference in responsibilities for the **LLM**, **System**, and **User**. In this section, we discuss each as separate roles: The LLM is the language model that helps process and interpret text from the User and System; the backend System relays messages, prompts, and API calls between the LLM and KG; and the User converses with the LLM without having to directly interact with the KG or System.

LLMs, Frameworks, and KGs Used: We tested variations of GPT [5], Code Llama [28], and Mistral [13] during the design of LinkQ, ultimately deciding on GPT-4. We found the performance of GPT-4 to be the most consistent, but any LLM (either off-the-shelf or fine-tuned) could be used in its place. Since KGs vary widely in their structure and quality [11], we use the Wikidata KG as it is well-maintained, generalizable, and has a robust API service.

3.1 Natural Language Question Interpretation

To start, the System instructs the LLM that: (1) it is responsible for generating KG queries from a User’s natural language questions; (2) it must help the user refine a vague, subjective, or open-ended question into a well-defined one; and (3) during refinement, it should only suggest data that it would expect to be in the KG.

Open-ended versus Targeted Question: The User might have an open-ended question, such as “*what are some interesting things about cats?*” When the User’s question is subjective or exploratory in nature, the LLM will follow up with suggested properties or relationships from the KG that could possibly be used in place of vague qualifiers like “*interesting*.” This back-and-forth ensures a well-formed KG query can be written (**G1**).

3.2 Finding IDs in the Knowledge Graph

LLMs are known to state plausible but erroneous information to users [27]. We observed this phenomenon during our initial work with various LLMs to generate KG queries – entity and relation IDs were often incorrect. An example is illustrated in Figure 2.

That said, it is computationally impractical to fine-tune an LLM to memorize all possible entity, relation, and property IDs in a KG. For example, the Wikidata KG has more than a hundred million entities [32]. Even if the KG were small in size, any updates to the data or IDs could result in having to retrain the LLM.

We implemented a protocol to address this, in which the LLM iteratively asks the System to fuzzy search for entities, find relevant properties, and traverse the KG to identify all the correct IDs required to generate a SPARQL query. The System makes requests

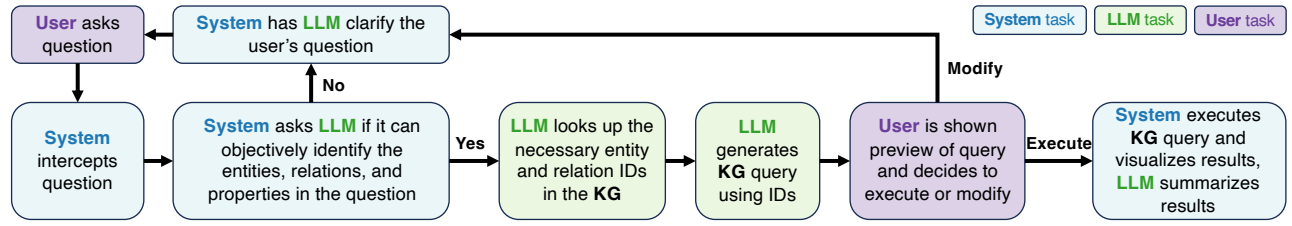


Figure 3: Illustrative state-machine of LinkQ’s system architecture, as described in Section 3. In our pipeline, the LLM, System, and User have distinct roles, responsible for communicating with one another to complete different tasks across different states.

to the various KG APIs, then passes the KG graph structure and ground-truth IDs to the LLM via system prompt messages (G2).

Fuzzy Searching for Entities by Name and Context: Our protocol uses a fuzzy search feature in the Wikidata API so the LLM does not have to rely on *exact* names, instead identifying entity IDs based on semantic similarity. The LLM can select the entity ID that most closely matches the context of the User’s question, which helps with duplicate and ambiguous entities – a common KG data quality and querying obstacle [19]. For example, if the User asks a question about the *box office total* for *The Godfather*, the LLM should select the entity ID for *The Godfather* that references the film, rather than the book series.

Given an Entity, Find its Relations and Properties: The LLM can ask the System to get all properties and relations for a given entity ID. The System requests data from Wikidata API, then responds with the results. Importantly, this ensures that the properties actually belong to that entity in the KG. For example, if the User asks about the *box office total* for *The Godfather*, the LLM should look for any properties or relations belonging to *The Godfather* that are named similarly to *box office*.

Traverse the Graph for Multi-Hop Relationships: To identify multi-hop relationships, the LLM can traverse the KG from one head entity to a tail entity, again by interfacing with the Wikidata API via the System. This is necessary for building successful queries, since KG data is maintained as a series of connected triplets. For example, when mapping data from the question “*Who are the Founders of Google, and what are their birthdates,*” the LLM would have to hop from the *Google* entity to its *Founder* entities to identify their *birthdate* properties.

3.3 Writing, Explaining, and Executing KG Queries

After all necessary IDs have been identified from the KG, the System instructs the LLM to generate a query to answer the original question. The KG query is then previewed, along with an LLM-generated explanation, to the User in the interface (discussed in Section 4). To produce this explanation, the System instructs the LLM to summarize why the generated query addresses the User’s original question, e.g., which components of the query have been mapped from the question. LinkQ takes roughly 10-15 seconds to complete the protocol of converting the User’s natural language question to an LLM-generated KG query and query explanation.

4 USER INTERFACE DESIGN

LinkQ is divided into three interface panels, shown in Figure 1: the Chat Panel, the Query Preview Panel, and the Query Results Panel.

4.1 Chat Panel

The Chat Panel (Figure 1-A) enables user communication with the LLM, maintains a record of the conversation, and optionally displays the full output generated by the LLM’s message passing with the System. This panel includes features to help simplify query

refinement (G1). For example, once a query is generated, two buttons are shown to the user: one to copy the query to the editor and another to copy and run the query immediately. This design prevents unintended overwriting of in-progress queries, although a query history is available for reference (see Section 4.2). Additional buttons are provided to insert convenient prompts into the chat, helping users correct or redirect the LLM as needed.

4.2 Query Preview Panel

The Query Preview Panel comprises the Query Editor, Entity-Relation Table, and the Query Graph visualization. These three pieces work in tandem to preview the potentially computationally expensive query before it is run, and ensure the accuracy of Wikidata entity identifiers (G3).

Query Editor The Query Editor (Figure 1-B1) is a standard SPARQL code interface with keyword highlighting, helping users assess the syntax of the query (G3). The editor also features a query history, allowing users to quickly retrieve past queries.

Entity-Relation Table The Entity-Relation Table (Figure 1-B2) provides contextual information (e.g., a human-readable label and short description) about the entity and property IDs found in the query (G3). This table is intended to help users assess whether the LLM has identified the correct IDs for the generated query.

Query Graph The Query Graph (Figure 1-B3) shows a visual preview (G3) of a syntactically valid query. This graphic allows the user to understand the relationships within their queries better, regardless of whether the queries are constructed by the LLM or themselves. Specifically, the query is parsed for basic graph patterns (BGP), and the triples contained within the BGPs are plotted within the graph. All items and data values are represented as nodes, and all properties are represented as edges. The nodes and edges are labeled by their corresponding text from the query, as well as labels retrieved from the KG. Known Wikidata entities are colored blue, and unresolved variables are colored orange.

4.3 Query Results Panel

The Query Result Panel (Figure 1-C) provides the results of the query in formats best for both consumption and analysis (G4).

Exportable Results Table Query results are automatically parsed from JSON to a cleaned CSV format, removing unnecessary meta-data retrieved from the KG, then visualized as a table. Users can choose to export the results for downstream analysis (G4).

LLM Results Summary The LLM also provides a succinct paragraph to summarize the query results for the user (G4). When the query results are empty, LinkQ instructs the LLM to provide a best guess for what could have gone wrong with the query.

5 DEMONSTRATION

To gather feedback on our system, we demonstrated its question-answering capabilities with a team of five data scientists (two we previously collaborated with, see Section 2) who work with KGs.

Use Case	Natural Language Question to Translate
Films	(Q1) Which directors won the <i>Academy Award for Best Director</i> between 2014 and 2024, and for which films? (Q2) Which film genres most commonly win the <i>Academy Award for Best Picture</i> ?
Cybersecurity	(Q3) What are some different types of cyberattacks? (Q4) What are examples of cyberattacks that have occurred in history?
Geography	(Q5) What is the official language for each country in Europe? (Q6) What are the top 10 tallest mountains in the world, and what country do they belong to?

Table 1: Questions we walked through with participants during the demonstration of LinkQ. More details in Section 5.

Our goal was to determine whether LinkQ satisfies its design goals, and to identify features that could be improved in the future.

We demoed three use cases with participants. All questions from the demonstration are provided in Table 1. Participants were also asked to input their own questions to LinkQ. Our supplemental provides complete details on our protocol, our participants’ experience levels, and the correct answers to each of the questions asked.

5.1 Question-Answering Results

Participants observed two inaccuracies. First, the LLM mistakenly used the ID for ‘*Best Director*’ instead of for ‘*Best Picture*,’ which participants identified using the Entity-Relation table (G3). This mistake was corrected after participants iterated with the LLM. In Q6, the query was syntactically correct but incorrectly resulted in U.S. mountains, likely due to a unit conversion error between feet and meters, as cautioned by the LLM’s Summary Panel.

During the free-form Q&A, participants intentionally asked questions they thought LinkQ would fail in translating. To their surprise, in 2/3 cases our system successfully resulted in syntactically correct SPARQL queries and correct IDs. Failures were due to missing data in Wikidata or incorrect unit specifications in queries.

Overall, participants praised LinkQ’s ability to preserve their semantic intent during question-answering (G1), and told us that – even when the LLM’s query failed, our Query Graph gave them the insight to update the query correctly themselves.

5.2 Participants’ Feedback on LinkQ

After the demonstration, we performed a semi-structured interview with participants to discuss the system features that seemed most helpful in answering questions, as well as which features could be added to improve usability. We highlight interesting observations and takeaways from participants to help inform this design space.

With enough guidance, LLMs can alleviate the burden of manual data retrieval: Without hesitation, the immediate feedback we received was, “*What I like about this is that I don’t have to write SPARQL queries anymore!*” Participants suggested it would be extremely beneficial to data scientists if our system could also retrieve data from multiple enterprise Wikis and databases simultaneously.

The KG-generated Entity-Relation table and Graph Query increase trust in the LLM’s outputs: All participants told us that knowing the table and query graph were extracted from the KG gave them confidence in the LLM’s queries (G2). The two participants who work most closely with KGs praised the graph query visualization: “*You see edge connections that you wouldn’t otherwise see from just the entity-relation table.*” Participants less familiar with KGs preferred the table, particularly the data descriptions.

The LLM Summary Panel can inform what went wrong with the query, or the query results: Participants told us, “*The LLM summary is helpful and informative... it gives us information on*

what could’ve gone wrong in the query. It captured the intention of my query, which was unexpected.” This feedback suggests that there are benefits to using LLMs over traditional NLP approaches for query building. Of course, the quality of an LLM’s insights depends on its own knowledge of the data domain.

Question-answering with ground truth data is seen as a novel task for LLMs: One participant told us, “*This is one of the cooler applications I’ve seen of LLMs. It’s retrieving actual, ground truth information. Usually LLMs are just text generators.*” Participants stressed again that research should continue to examine how LLMs can be used to improve search and data retrieval tasks for data scientists, as manual keyword search is both tedious and flawed.

Make visual distinctions between LLM output and KG output: One participant told us that he sometimes forgot which outputs came from the LLM versus the KG, and suggested that a visual distinction (e.g., different colored background panels) should be added. All participants agreed that both the LLM summary and KG results table seem equally important, just categorically different.

Additional visualizations seemed unnecessary: We asked participants if they wanted or expected to see additional visualizations after receiving query results. Surprisingly, not a single participant said yes, and instead told us the LLM’s text summary and KG results table were ideal (G4). Two participants told us that it depends on the data and task, e.g., an organizational data domain could benefit from hierarchical-based charts.

6 DISCUSSION AND FUTURE WORK

The positive feedback we received from participants suggests that systems like LinkQ can be successfully used for question-answering over knowledge graphs. That said, it is important to note that we observed a great deal of necessary hand-holding in implementing an LLM for successful KG query writing. Of course, this makes sense, as off-the-shelf LLMs are typically performant on text-based tasks like summarization [34]. Future work can investigate optimizing our approach (Section 3) with this in mind.

LinkQ’s Protocol versus RAG: Retrieval augmented generation (RAG) [16] approaches use text embeddings to compare the user’s question with a document knowledge base, and inject the most relevant documents into a prompt to improve the LLM’s response. In contrast, LinkQ instructs the LLM to iteratively explore the KG data (with the help of the System) to find the IDs it needs to generate a query. In the future, we would like to quantitatively evaluate the effectiveness of LinkQ against traditional RAG and NLP approaches. We can quantitatively compare the goodness of generated queries, as well as users’ feedback on how well the LLM captures their semantic intent during question-answering. Altogether, these directions can help us better understand how systems like LinkQ can aid data scientists in conducting visual data analysis with LLMs.

7 CONCLUSION

We presented LinkQ, a system that implements an LLM for KG question-answering without the expertise of a KG querying language. LinkQ supports users in asking natural language questions, either open-ended or targeted, which are iteratively refined with an LLM until they can be translated into well-formed KG queries. LinkQ implements a scalable protocol to reduce the frequency of LLMs hallucinating data from the KG, requiring no fine-tuning of the LLM. Our interface provides two types of query visualizations, an entity-relation table and query graph structure, to help users both ascertain the accuracy of the LLM’s generated query, and to inform them of how a query could be incorrect or improved upon. From a demonstration with five data scientists, we found that LinkQ exceeded expectations on converting NL questions to SPARQL queries, which participants stressed is a huge improvement to the burden of manually writing and assessing KG queries.

ACKNOWLEDGMENTS

We thank our collaborators at MIT Lincoln Laboratory for their helpful feedback on LinkQ, as well as the reviewers for their input in improving the quality of our paper.

DISTRIBUTION STATEMENT A. Approved for public release. Distribution is unlimited. This material is based upon work supported by the Combatant Commands under Air Force Contract No. FA8702-15-D-0001. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Combatant Commands. © 2024 Massachusetts Institute of Technology. Delivered to the U.S. Government with Unlimited Rights, as defined in DFARS Part 252.227-7013 or 7014 (Feb 2014). Notwithstanding any copyright notice, U.S. Government rights in this work are defined by DFARS 252.227-7013 or DFARS 252.227-7014 as detailed above. Use of this work other than as specifically authorized by the U.S. Government may violate any copyrights that exist in this work.

REFERENCES

- [1] Resource Description Framework (RDF) Model and Syntax Specification — w3.org. <https://www.w3.org/TR/PR-rdf-syntax/Overview.html>. [Accessed 29-07-2024]. 2
- [2] Z. Abbasianteab, Y. Yuan, E. Kanoulas, and M. Aliannejadi. Let the llms talk: Simulating human-to-human conversational qa via zero-shot llm-to-llm interactions. *WSDM '24*, p. 8–17. ACM, New York, NY, USA, 2024. doi: 10.1145/3616855.3635856 1
- [3] B. Abu-Salih. Domain-specific knowledge graphs: A survey. *J. Netw. Comput. Appl.*, 185:103076, 2021. doi: 10.1016/j.jnca.2021.103076 1
- [4] G. Agrawal, T. Kumarage, Z. Alghamdi, and H. Liu. Can knowledge graphs reduce hallucinations in LLMs? : A survey. In K. Duh, H. Gomez, and S. Bethard, eds., *Proc. NAACL: Hum. Lang. Tech. (Vol. 1: Long Papers)*, pp. 3947–3960. Association for Computational Linguistics, Mexico City, Mexico, June 2024. doi: 10.18653/v1/2024.naacl-long.219 2
- [5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In *Adv. Neural Inf.*, vol. 33, pp. 1877–1901, 2020. 2
- [6] B. Ell, D. Vrandečić, and E. Simperl. Spartiquation: Verbalizing sparql queries. In *Proc. ESWC*, pp. 117–131. Springer, 2013. 1, 2
- [7] R. C. Fernandez, A. J. Elmore, M. J. Franklin, S. Krishnan, and C. Tan. How large language models will disrupt data management. *Proc. VLDB*, 16(11):3302–3309, 2023. 2
- [8] S. Ferré. Sparklis: An expressive query builder for sparql endpoints with guidance in natural language. *Semant. Web*, 8(3):405–418, 2017. doi: 10.3233/SW-150208 1, 2
- [9] P. Graftkin, M. Mironov, M. Fellmann, B. Lantow, K. Sandkuhl, and A. V. Smirnov. Sparql query builders: Overview and comparison. In *BIR Workshops*, pp. 255–274, 2016. 1, 2
- [10] Q. Guo, F. Zhuang, C. Qin, H. Zhu, X. Xie, H. Xiong, and Q. He. A survey on knowledge graph-based recommender systems. *IEEE Trans. Knowl. Data Eng.*, 34(08):3549–3568, 2022. doi: 10.1109/TKDE.2020.3028705 1
- [11] A. Hogan, E. Blomqvist, M. Cochez, C. D’amato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann. Knowledge graphs. *ACM Comput. Surv.*, 54(4), 2021. doi: 10.1145/3447772 1, 2
- [12] J. Huang, Y. Xi, J. Hu, and J. Tao. Flownl: Asking the flow data in natural languages. *IEEE Trans. Vis. Comput. Graph.*, 29(1):1200–1210, 2023. doi: 10.1109/TVCG.2022.3209453 2
- [13] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. Mistral 7b, 2023. 2
- [14] N. Karanikolas, E. Manga, N. Samaridi, E. Tousidou, and M. Vassilakopoulos. Large language models versus natural language understanding and generation. In *In Proc. PCI*, pp. 278–290, 2023. 1
- [15] A. Khan. Knowledge graphs querying. *SIGMOD Rec.*, 52(2):18–29, aug 2023. doi: 10.1145/3615952.3615956 1
- [16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proc. NIPS*. Red Hook, NY, USA, 2020. 4
- [17] H. Li, G. Appleby, C. D. Brumar, R. Chang, and A. Suh. Knowledge graphs in practice: Characterizing their users, challenges, and visualization opportunities. *IEEE Trans. Vis. Comput. Graph.*, 30(1):584–594, 2024. doi: 10.1109/TVCG.2023.3326904 1
- [18] H. Li, Y. Wang, S. Zhang, Y. Song, and H. Qu. Kg4vis: A knowledge graph-based approach for visualization recommendation. *IEEE Trans. Vis. Comput. Graph.*, 28(01):195–205, 2022. doi: 10.1109/TVCG.2021.3114863 1
- [19] M. Lissandrini, D. Mottin, K. Hose, and T. B. Pedersen. Knowledge graph exploration systems: are we lost? In *CIDR*, vol. 22, pp. 10–13, 2022. 1, 3
- [20] M. Lissandrini, D. Mottin, T. Palpanas, and Y. Velegrakis. Graph-query suggestions for knowledge graph exploration. In *Proc. ACM WWW*, pp. 2549–2555, 2020. 1
- [21] A. Martino, M. Iannelli, and C. Truong. Knowledge injection to counter large language model (llm) hallucination. In *European Semantic Web Conference*, pp. 182–185. Springer, 2023. 2
- [22] R. Mitra, A. Narechania, A. Endert, and J. Stasko. Facilitating conversational interaction in natural language interfaces for visualization. In *Proc. VIS*, pp. 6–10, 2022. doi: 10.1109/VIS54862.2022.00010 2
- [23] A.-C. Ngonga Ngomo, L. Bühmann, C. Unger, J. Lehmann, and D. Gerber. Sorry, i don’t speak sparql: translating sparql queries into natural language. In *Proc. ACM WWW*, p. 977–988. ACM, New York, NY, USA, 2013. doi: 10.1145/2488388.2488473 1
- [24] OpenAI. OpenAI Current Models. <https://platform.openai.com/docs/models/>. Accessed: 2024-08-05. 2
- [25] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Trans. Knowl. Data Eng.*, pp. 1–20, 2024. doi: 10.1109/TKDE.2024.3352100 2
- [26] J. C. Rangel, T. M. de Farias, A. C. Sima, and N. Kobayashi. Sparql generation: an analysis on fine-tuning openllama for question answering over a life science knowledge graph. *arXiv preprint arXiv:2402.04627*, 2024. 2
- [27] V. Rawte, A. Sheth, and A. Das. A survey of hallucination in large foundation models. *arXiv preprint arXiv:2309.05922*, 2023. 2
- [28] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bittan, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve. Code llama: Open foundation models for code, 2024. 1, 2
- [29] A. Srinivasan and V. Setlur. Snowy: Recommending utterances for conversational visual analysis. In *ACM UIST*, pp. 864–880, 2021. 2
- [30] N. Sultanum and A. Srinivasan. Datatales: Investigating the use of large language models for authoring data-driven articles. In *Proc. VIS*, pp. 231–235, 2023. 2
- [31] Y. Wen, Z. Wang, and J. Sun. Mindmap: Knowledge graph prompting sparks graph of thoughts in large language models. In *Proc. Association for Computational Linguistics*, 2024. 2
- [32] Wikipedia. Wikidata Statistics. <https://www.wikidata.org/wiki/Wikidata:Statistics>. Accessed: 2024-04-24. 2
- [33] A. Yamaguchi, K. Kozaki, K. Lenz, H. Wu, and N. Kobayashi. An intelligent sparql query builder for exploration of various life-science databases. In *IESD@ ISWC*, 2014. 1
- [34] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, and X. Hu. Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Trans. Knowl. Discov. Data*, feb 2024. Just Accepted. doi: 10.1145/3649506 1, 4